

Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

División Ciencias de la Ingeniería

Software Avanzado

Eiler Rigoberto Gómez Figueroa

202031693

Ensayo OAuth 2.0 + OIDC vs SAML



Ing. Moises Granados

Quetzaltenango, Agosto del 2026

OAuth 2.0 + OIDC vs SAML

Pensemos en una empresa mediana: sus empleados usan el correo corporativo, un sistema de recursos humanos, una herramienta de gestión de proyectos y una plataforma de videoconferencias. Sin ningún mecanismo especial, cada uno de esos sistemas tendría su propio usuario y contraseña, obligando al empleado a recordar cuatro credenciales distintas y a la empresa a mantener cuatro bases de datos de usuarios separadas. Si alguien roba la contraseña de uno de esos sistemas, tiene acceso a ese sistema. Pero si alguien roba la contraseña maestra del empleado, potencialmente accede a todos.

La solución a este problema se llama delegación de identidad: en lugar de que cada aplicación verifique por su cuenta quién es el usuario, todas confían en un único sistema central que ya lo hizo. Ese sistema central se llama proveedor de identidad, o IdP por sus siglas en inglés. El resultado visible para el usuario es lo que conocemos como Single Sign-On, o SSO: entras una sola vez y ya estás dentro de todo.

Para que esto funcione de manera estandarizada e interoperable entre sistemas de diferentes fabricantes, existen dos protocolos principales: OAuth 2.0 junto con OpenID Connect (OIDC), y SAML 2.0. Este ensayo explica qué es cada uno, cómo funcionan, cuáles son sus diferencias en seguridad, y cuándo conviene usar uno u otro.

¿Por qué no simplemente compartir contraseñas?

Antes de que existieran estos estándares, cuando una aplicación necesitaba acceder a datos de otra en nombre del usuario, la solución común era simple y peligrosa: el usuario le daba su contraseña a la aplicación para que esta la usara. Imagina darle la llave de tu casa a alguien para que recoja tu correspondencia: funciona, pero esa persona tiene acceso a todo, no solo al buzón.

El RFC 6749, el documento oficial que define OAuth 2.0, describe exactamente este problema:

"Las aplicaciones de terceros deben almacenar las credenciales del propietario del recurso para su uso futuro, normalmente una contraseña en texto plano [...] Los propietarios de recursos no pueden revocar el acceso a un tercero individual sin revocar el acceso a todos los terceros." (Hardt, 2012, RFC 6749)

En otras palabras: si una de esas aplicaciones es comprometida, el atacante obtiene la contraseña real del usuario y puede usarla en cualquier otro sistema. OAuth resuelve esto de raíz: en lugar de compartir la contraseña, el usuario autoriza a la aplicación a recibir un token, que es como un pase de acceso temporal y limitado. Si ese pase es robado o ya no se necesita, se cancela sin afectar a nada más.

OAuth 2.0: el protocolo que dice "puedes hacer esto en mi nombre"

OAuth 2.0 es un protocolo de autorización. La palabra clave es "autorización": se trata de permisos, no de identidad. Cuando usas "Iniciar sesión con Google" en una aplicación externa, lo que ocurre es que Google le dice a esa aplicación: "Este usuario existe y nos autorizó a darle acceso a su nombre y correo". La aplicación nunca ve tu contraseña de Google.

El proceso ocurre en varios pasos. La aplicación redirige al usuario al IdP (Google, en el ejemplo). El usuario se autentica ahí y aprueba qué información desea compartir. El IdP devuelve un código temporal a la aplicación. La aplicación intercambia ese código por un token de acceso, que es la llave con la que podrá hacer peticiones en nombre del usuario. Este intercambio ocurre en un canal seguro entre servidores, nunca expuesto en la pantalla del usuario. La variante más segura de este proceso se llama Authorization Code Flow con PKCE, y es la recomendada actualmente para cualquier aplicación.

Sin embargo, OAuth solo resuelve la autorización. No responde a la pregunta: "¿Quién es exactamente este usuario?". Para eso existe OpenID Connect.

OpenID Connect: la capa que añade identidad a OAuth

OpenID Connect, o OIDC, es una extensión construida sobre OAuth 2.0 que agrega una respuesta estándar a la pregunta de identidad. La especificación oficial lo describe así:

"OpenID Connect 1.0 es una capa de identidad sencilla que se ejecuta sobre el protocolo OAuth 2.0. Permite a los clientes verificar la identidad del usuario final basándose en la autenticación realizada por un servidor de autorización." (OpenID Foundation, 2014, OpenID Connect Core 1.0)

En la práctica, cuando una aplicación usa OIDC, además del token de acceso que ya provee OAuth, recibe un segundo token llamado ID token. Este ID token es un documento pequeño y firmado que contiene información concreta del usuario: su identificador único, su correo electrónico, su nombre, y cuándo fue autenticado, entre otros datos. La aplicación puede leer ese documento y saber exactamente con quién está tratando, sin necesidad de consultar ninguna base de datos adicional.

La combinación OAuth 2.0 + OIDC es hoy el estándar dominante para SSO en aplicaciones modernas. La soportan de forma nativa Google, Microsoft, GitHub, Apple y prácticamente cualquier plataforma de identidad contemporánea.

SAML 2.0: el estándar que vive en las empresas

SAML 2.0 es un estándar más antiguo, ratificado en 2005 por OASIS (una organización internacional de estándares). Fue diseñado específicamente para entornos empresariales donde múltiples organizaciones necesitan compartir identidades de forma confiable, por ejemplo cuando los empleados de una empresa contratante necesitan acceder a sistemas del cliente sin crear una cuenta nueva en cada uno.

En SAML, el mecanismo central no son tokens JSON sino aserciones XML. Una aserción es básicamente una declaración formal y firmada del IdP que dice: "Yo, el proveedor de identidad de esta empresa, certifico que este usuario es quien dice ser, se autenticó a las 10:32 del día de hoy

y tiene los siguientes atributos". Esa declaración viaja firmada digitalmente con un certificado criptográfico, lo que garantiza que nadie la alteró en el camino.

El flujo es el siguiente: el usuario intenta acceder a una aplicación (llamada Service Provider o SP). La aplicación le dice: "No te conozco, ve a autenticarte con tu IdP". El usuario va al IdP, se autentica, y el IdP genera esa aserción XML firmada. El navegador del usuario actúa como mensajero y lleva esa aserción de regreso a la aplicación, que verifica la firma y concede acceso. Para que la aplicación sepa qué certificado usar para verificar la firma, IdP y SP intercambian previamente un documento llamado metadata, que contiene esa configuración.

Seguridad en ambos enfoques

Tanto OAuth/OIDC como SAML son seguros si se implementan bien. Sus vulnerabilidades vienen de ángulos distintos.

En OAuth/OIDC el riesgo principal está en dónde se guarda el token. Si una aplicación web guarda el token en el almacenamiento local del navegador (localStorage), cualquier script malicioso que logre ejecutarse en esa página puede robarlo. Esto se conoce como ataque XSS. La mitigación es guardar el token en una cookie con la bandera HttpOnly, que le impide al JavaScript del navegador acceder a ella. Otro riesgo es que un atacante intercepte el proceso de autorización engañando a la aplicación con una URL de redirección falsa; el mecanismo PKCE y el parámetro state existen precisamente para prevenir eso.

En SAML el riesgo más conocido es el XML Signature Wrapping. Sin entrar en tecnicismos: SAML usa XML, y el XML es un formato complicado de procesar de forma segura. Un atacante puede construir un documento XML que tenga una firma válida en una parte, pero con datos maliciosos en otra parte que el sistema termina usando sin darse cuenta. La solución es que el sistema que recibe la aserción sea muy estricto en verificar exactamente qué parte del documento fue firmada. La guía de seguridad de OWASP para SAML recomienda también usar tiempos de

vida cortos en las aserciones y rechazar cualquier aserción que ya haya sido usada anteriormente, para evitar ataques de repetición.

¿Cuándo usar cada uno?

La diferencia práctica es sencilla: OAuth 2.0 + OIDC es la elección para todo lo nuevo. Aplicaciones web modernas, aplicaciones móviles, APIs, servicios en la nube y cualquier sistema construido en los últimos años prácticamente asumen OIDC como base. Es más fácil de implementar, los tokens son más livianos y el ecosistema de herramientas es enorme.

SAML 2.0 sigue siendo necesario cuando hay que integrarse con sistemas empresariales ya existentes que hablan ese protocolo. Grandes plataformas como Salesforce, SAP, o los sistemas de identidad corporativa basados en Microsoft Active Directory Federation Services (ADFS) tienen SAML profundamente integrado. Reemplazarlo implicaría migrar esos sistemas, lo cual en entornos empresariales puede ser inviable.

En la práctica, muchas organizaciones usan ambos al mismo tiempo. Herramientas como Okta, Auth0 o Microsoft Azure Active Directory actúan como un puente: el empleado se autentica una sola vez, y ese proveedor de identidad central habla SAML con los sistemas viejos y OIDC con los nuevos, sin que el usuario note diferencia.

Conclusiones

- OAuth 2.0 + OIDC y SAML 2.0 son dos respuestas al mismo problema: cómo permitir que un usuario acceda a múltiples sistemas sin tener que autenticarse en cada uno, y sin que ninguno de esos sistemas tenga que conocer su contraseña real.
- OAuth 2.0 resolvió el problema de la autorización delegada: permitir que una aplicación actúe en nombre de un usuario con permisos limitados. OIDC completó esa solución añadiendo una forma estándar de transmitir la identidad del usuario. Juntos conforman el estándar moderno para SSO en internet.

- SAML 2.0, más antiguo y más complejo, sigue siendo el estándar del mundo empresarial. Su fortaleza es la interoperabilidad entre organizaciones distintas y la madurez de su adopción en sistemas corporativos. Su debilidad es la complejidad técnica que conlleva trabajar con XML firmado.

Referencias

- Hardt, D. (Ed.). (2012). *RFC 6749: The OAuth 2.0 Authorization Framework*. IETF.
- Jones, M., Bradley, J., & Sakimura, N. (2015). *RFC 7519: JSON Web Token (JWT)*. IETF.
- OpenID Foundation. (2014). *OpenID Connect Core 1.0*.
- OASIS. (2005). *SAML V2.0 Technical Overview*.
- OWASP Foundation. (2024). *SAML Security Cheat Sheet*.
- OWASP Foundation. (2024). *OAuth 2.0 Cheat Sheet*.

Links de interés

- <https://datatracker.ietf.org/doc/html/rfc6749>
- https://openid.net/specs/openid-connect-core-1_0.html
- [SAML 2.0 Technical Overview | OASIS Standard](#)
- https://cheatsheetseries.owasp.org/cheatsheets/SAML_Security_Cheat_Sheet.html
- https://cheatsheetseries.owasp.org/cheatsheets/OAuth2_Cheat_Sheet.html